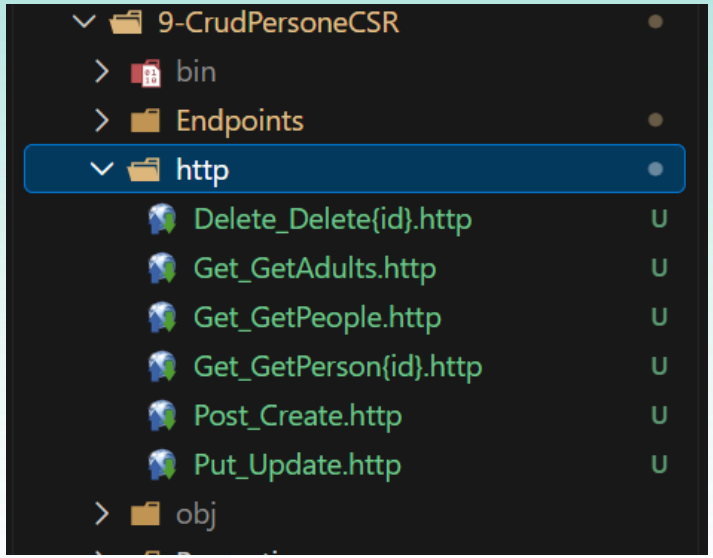
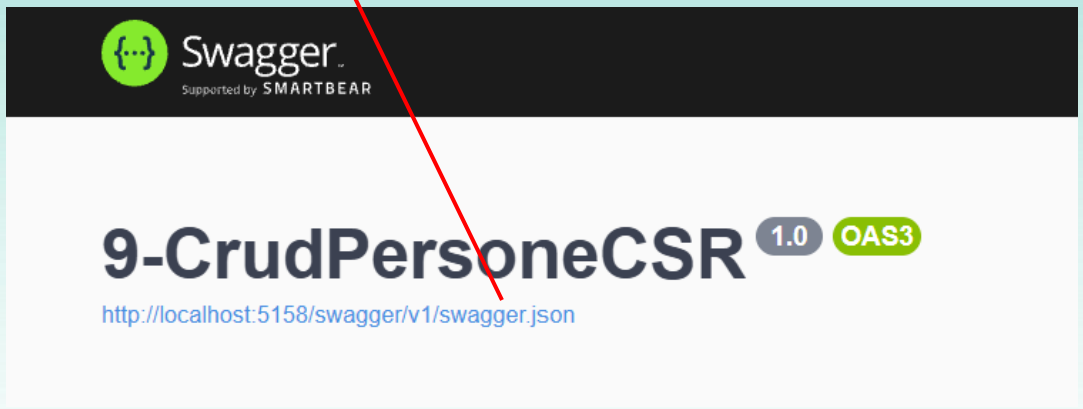
Esercitazione8 – Web API Person

Generare i file .http in automatico

Non serve farli a mano, ma basta generarli dalla descrizione OpenAPI (Swagger/Scalar) dell'API.

- Installa lo strumento (una sola volta): dotnet tool install --global httpgenerator
- Avviare l'API: dotnet run
- Lancia il comando da powershell nel percorso del progetto:
  `httpgenerator http://localhost:XXXX/swagger/v1/swagger.json --output-type OneRequestPerFile --output ./http`
- Usare `OneFile` anziché `OneRequestPerFile` se si vuole un unico file .http che ingloba tutte le richieste http.





Esercitazione8.1 – Web API Piloti

ORA TOCCA A VOI!

Un'API CRUD come quella delle persone, ma per i piloti di F1.

- Modello da eseguire: `\Exercises\8.1-CrudPiloti`
- Consegna completa nel `README`
- Si lavora in `\MyExercises\8.1-CrudPiloti`



COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione9 – CSR

Cerchiamo di dare un’architettura pulita alla nostra applicazione.

Quindi partendo dall’Esercitazione8, la nuova applicazione avrà le stesse funzionalità precedenti, ma la struttura del progetto verrà modificata in modo da rispettare la CLEAN ARCHITECTURE.

Oggi nel mondo del software è molto diffuso l’uso delle cosiddette Clean Architecture. Vengono chiamate così perché si basano tutte sullo stesso principio di progettazione del software: la separazione delle responsabilità.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione9 – CSR

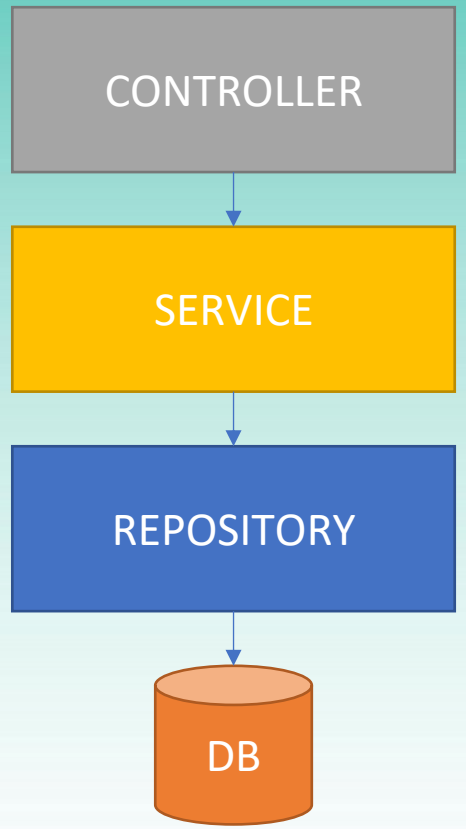
MODELLO CSR

Nello sviluppo software, creare applicazioni scalabili, manutenibili e testabili è fondamentale.

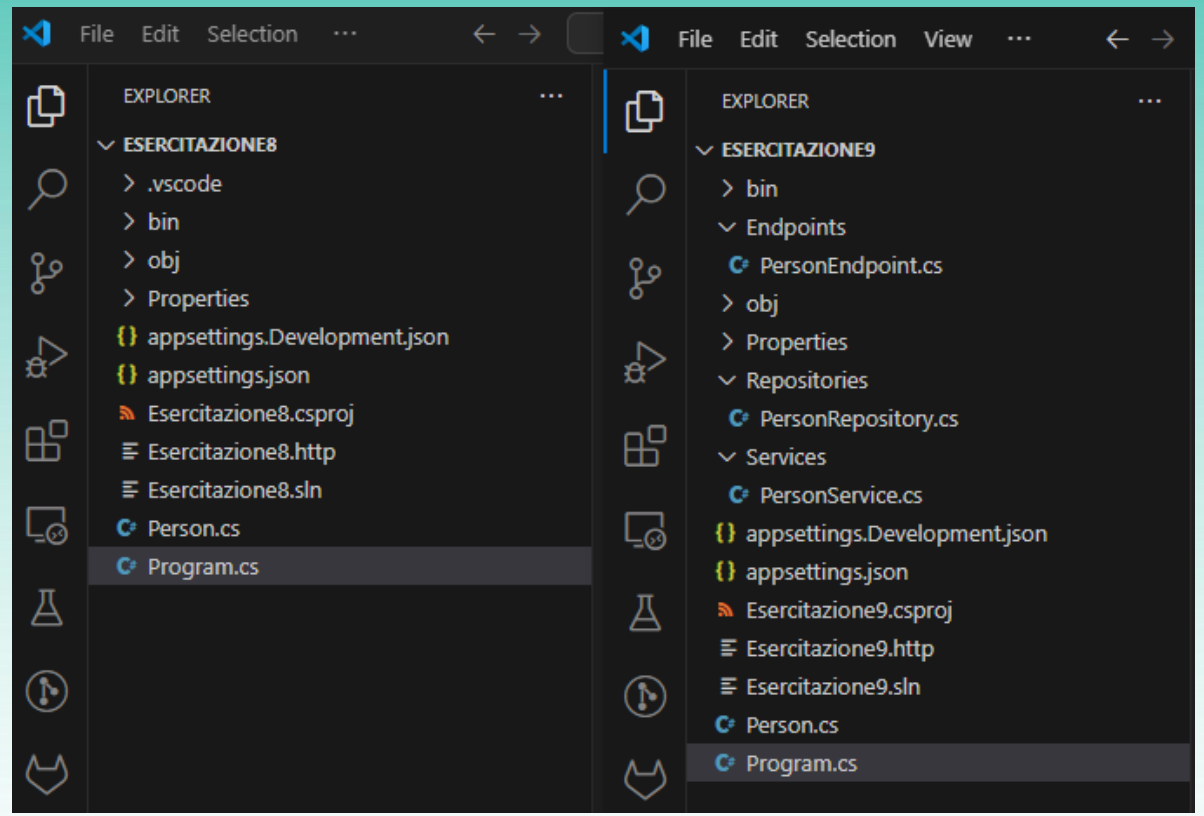
Un modello architettonico che aiuta a raggiungere questi obiettivi è il modello Controller-Service-Repository (CSR). Questo modello promuove la separazione delle preoccupazioni dividendo la logica dell'applicazione in tre livelli distinti: il Controller, il Service e il Repository.

Ogni livello ha una responsabilità specifica, rendendo il codice più facile da gestire, testare e scalare:

- il livello Controller gestisce le richieste HTTP in arrivo, le elabora e restituisce le risposte HTTP appropriate. Agisce come intermediario tra il client e il livello Service;
- il livello Service contiene la logica aziendale dell'applicazione. Elabora i dati ricevuti dal controller, esegue le operazioni necessarie e comunica con il livello di Repository per recuperare o rendere persistenti i dati;
- il livello Repository, gestisce l'accesso ai dati e la persistenza. Comunica con il database o altre fonti di dati, incapsulando la logica per interrogare, salvare e aggiornare i dati.



Esercitazione9 – CSR



Differenze fra Esercitazione8 e Esercitazione9

Mettiamo a confronto l'architettura delle due applicazioni:

- Nell'Esercitazione8 avevamo creato una solo classe Person all'interno del file Person.cs e endpoint, logica dell'applicazione e accesso ai dati avvenivano all'interno del Program.cs;
- Nell'Esercitazione9 sono invece presenti altre cartelle che permettono di rispettare la separazione delle responsabilità:
    1. Il livello Controller lo troviamo all'interno del file PersonEndpoint.cs della cartella Endpoints;
    2. Il livello Service lo troviamo all'interno del file PersonService.cs della cartella Services;
    3. Il livello Repository lo troviamo all'interno del file PersonRepository.cs della cartella Repositories.

Le cartelle Endpoints, Services e Repositories ci permettono di suddividere le responsabilità e di farlo per ogni singolo modello (classe).
Quindi, se oltre alla classe Person di Person.cs in futuro aggiungeremo un'altra classe (ad esempio la classe Animal in un file Animal.cs), aggiungeremo altri file nelle cartelle Endpoints, Services e Repositories: esattamente un ulteriore file per ogni livello di responsabilità, cioè per ogni cartella (rispettivamente i files AnimalEndpoint.cs, AnimalService.cs e AnimalRepository.cs).

Esercitazione9 – CSR

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

app.MapAPIEndpoints();

// Configure the HTTP request pipeline.

if (app.Environment.IsDevelopment())

{

    app.UseSwagger();

    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.Run();
```

Program.cs

Rispetto all'Esercitazione8, nel Program.cs, togliamo tutto il codice che appartiene ai livelli di responsabilità CSR, per spostarli in altri files.
Ricordiamo però che il Program.cs è il punto di avvio dell'applicazione ed è quello che si occupa di creare ed eseguire l'app.
E' quindi necessario dire all'app di aggiungere le nostre API, altrimenti non verrano create.
Per farlo richiamiamo un metodo, statico, creato da noi: MapAPIEndpoints() che contiene la definizione delle API.
Vediamo con la slide successiva la definizione del metodo MapAPIEndpoints() della classe PersonEndpoints.

Esercitazione9 – CSR

```
public static class PersonEndpoints

{

    private static PersonService personService = new
PersonService();

    public static void MapAPIEndpoints(this
IEndpointRouteBuilder app)

    {

        app.MapGet("/getPeople", () =>
personService.GetPeople());

        app.MapGet("/getPerson/{id}", (int id) =>
personService.GetPerson(id));

        app.MapPost("/create", (Person person) =>

        {

            bool result =
personService.Create(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.BadRequest();

        });
```

```
        app.MapPut("/update", (Person person) =>

        {

            bool result =
personService.Update(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

        app.MapDelete("/delete/{id}", (int id) =>

        {

            bool result = personService.Delete(id);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

    }

}
```

PersonEndpoints.cs
Il file contiene la definizione di una classe statica: PersonEndpoints.

Questa classe è quella che deve contenere il livello Controller e quindi la definizione delle API, ma non deve contenere le responsabilità del livello Service e del livello Repository.

Ricordiamo che una classe statica non può essere istanziata e può contenere solo membri statici.

La classe PersonEndpoints ha:

- Un istanza statica di PersonService, che come vedremo è una classe che contiene il livello Service;
- Il metodo statico MapAPIEndpoints, che contiene la definizione delle API e le aggiunge all'app. Dentro le API non troviamo logica. Ogni API utilizza il membro statico personService della classe PersonEndpoints per richiamare i metodi della classe PersonService.

Ogni API della classe PersonEndpoints quindi richiama metodi del Service, prende la risposta e restituisce una risposta al chiamante (client).

Vediamo con la slide successiva la classe PersonService.

Esercitazione9 – CSR

```
public class PersonService

{

    private PersonRepository personRepository = new
PersonRepository();

    public IEnumerable<Person> GetPeople()

    {

        return personRepository.GetPeople();

    }

    public Person? GetPerson(int id)

    {

        return personRepository.GetPerson(id);

    }

    public bool Create(Person person)

    {

        return personRepository.Create(person);

    }

    public bool Update(Person person)

    {

        return personRepository.Update(person);

    }

    public bool Delete(int id)

    {

        return personRepository.Delete(id);

    }

}
```

PersonService.cs
Il file contiene la definizione di una classe: PersonService.

Questa classe è quella che deve contenere il livello Service e quindi la logica aziendale dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Repository.

La classe PersonService ha:

- Un istanza di PersonRepository, che come vedremo è una classe che contiene il livello Repository;
- La definizione dei metodi richiamati nelle API della classe PersonEndpoints. Questi metodi attualmente non contengono logica, nel nostro caso non stiamo manipolando i dati che ci arrivano in ritorno dal livello Repository, ma avremmo dovuto farlo qui, se necessario. Dentro i metodi non effettuiamo l'accesso diretto ai dati, perché ciò aspetta al livello Repository, infatti ogni metodo della classe PersonService utilizza il membro personRepository della classe PersonService per richiamare i metodi della classe PersonRepository.

Ogni metodo della classe PersonService quindi richiama metodi del Repository, prende la risposta e, dopo aver inserito la logica dell'applicativo, se presente, restituisce una risposta al chiamante (API del livello Controller).

Vediamo con la slide successiva la classe PersonRepository.

Esercitazione9 – CSR

```
public class PersonRepository

{

    private Dictionary<int, Person> dict = new Dictionary<int, Person>();

    public IEnumerable<Person> GetPeople()

    {

        if(dict.Count > 0)

            return dict.Values.ToArray();

        else

            return [];

    }

    public Person? GetPerson(int id)

    {

        if(dict.Count > 0 && dict.ContainsKey(id))

            return dict[id];

        else

            return null;

    }

    public bool Create(Person person)

    {

        if (dict.Count > 0 && dict.ContainsKey(person.id))

            return false;

        else

        {

            dict.Add(person.id, person);

            return true;

        }

    }
```

```
    public bool Update(Person person)

    {

        if ((dict.Count > 0 && !dict.ContainsKey(person.id)) || dict.Count == 0)

            return false;

        else

        {

            dict[person.id] = person;

            return true;

        }

    }

    public bool Delete(int id)

    {

        if ((dict.Count > 0 && !dict.ContainsKey(id)) || dict.Count == 0)

            return false;

        else

        {

            dict.Remove(id);

            return true;

        }

    }

}
```

PersonRepository.cs
Il file contiene la definizione di una classe: PersonRepository.

Questa classe è quella che deve contenere il livello Repository e quindi l'accesso ai dati dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Service (già definito in PersnService).

La classe PersonRepository ha:

- Un istanza di un Dictionary, che simula il nostro DB;
- La definizione dei metodi richiamati nel nella classe PersonService. Questi metodi si occupano di accedere, leggere e scrivere nei dati, che nel nostro caso sono rappresentati da dict. Ogni metodo della classe PersonRepository quindi prende i dati dal DB e li restituisce ai Controller, dopo aver inserito la logica dell'applicativo.

Ogni metodo della classe PersonRepository quindi si occupa di leggere e scrivere nei dati (nel nostro caso dict), e restituisce una risposta al chiamante (metodi del livello Service).

Esercitazione9.1 – Web API Piloti CSR

SIGNORI…IO MI RIPOSO TOCCA A VOI!



Riorganizzate l'API 8.1-CrudPiloti in Controller – Service – Repository (in gergo, fate il *refactoring*), come abbiamo fatto con le persone nell'esercitazione 9.

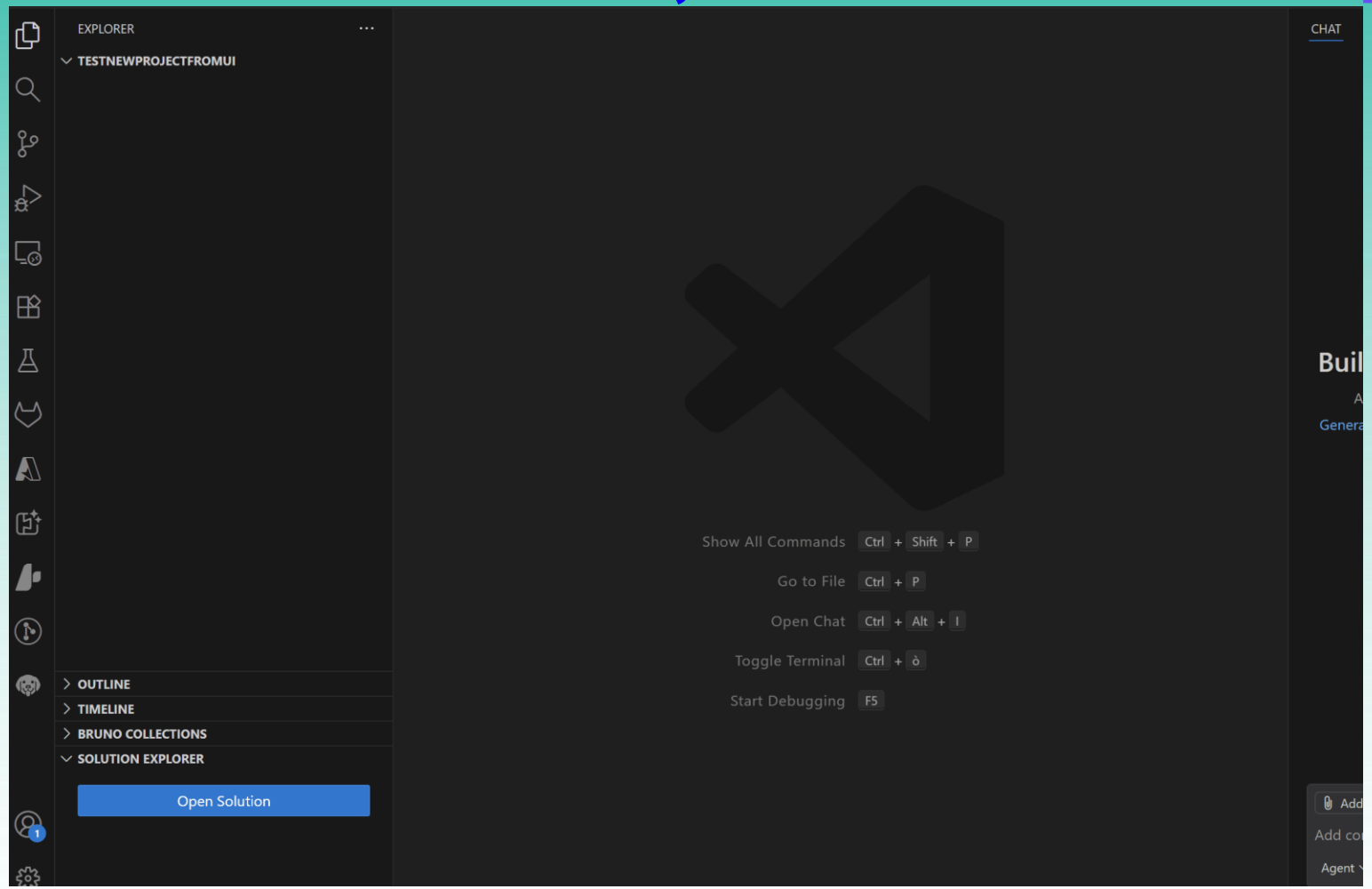
- Modello da eseguire: `\Exercises\9.1-CrudPiloti`
- Consegna completa nel `README`
- Si lavora in `\MyExercises\9.1-CrudPilotiCSR`

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Gestione della Soluzione in .NET

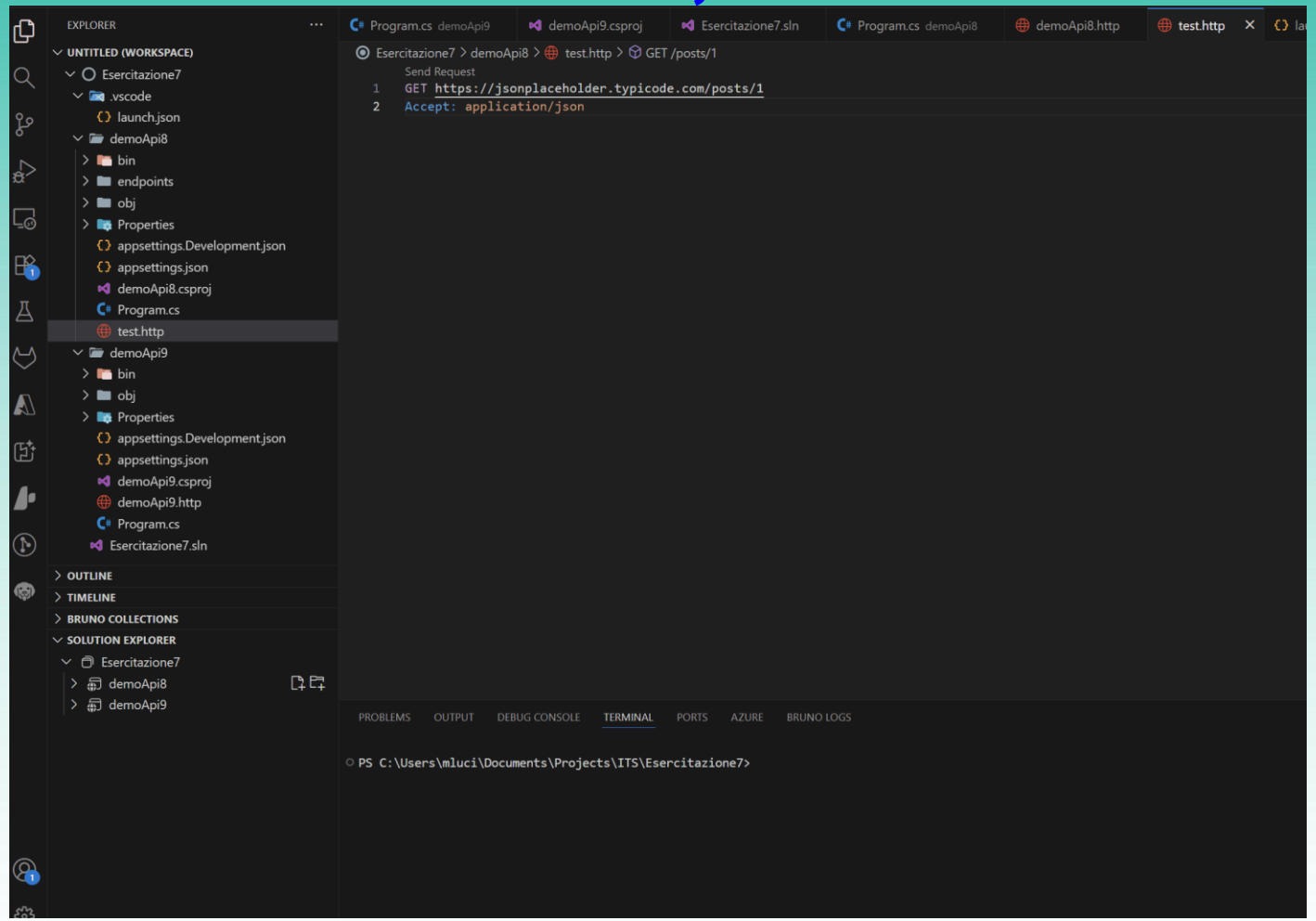
- Creare una nuova soluzione vuota
  ```
  dotnet new sln -n NomeSoluzione
  ```
- Creazione dei progetti
  ```
  dotnet new console -n ProgettoConsole
  dotnet new webapi -n ProgettoWebAPI
  ```
- Aggiunta dei progetti alla soluzione
  ```
  dotnet sln NomeSoluzione.sln add ./ProgettoConsole/ProgettoConsole.csproj
  dotnet sln NomeSoluzione.sln add ./ProgettoWebAPI/ProgettoWebAPI.csproj
  ```
- Verifica dei progetti inclusi nella soluzione
  ```
  dotnet sln NomeSoluzione.sln list
  ```

Addio terminale, benvenuta GUI 🙏



1. CTRL+SHIFT+P
2. > .NET New Project
3. Cercare il template progetto desiderato (es. webapi, console…)
4. Seguire i passi guidati
5. In «Show all template options» sarà possibile modificare la versione del framework, come anche altre impostazioni che possono tornare utili (es. debug…)

Addio terminale, benvenuta GUI 🙏



In ‘Solution Explorer’ si ha la possibilità di gestire la soluzione in modo estremamente semplificato come:

- Aggiungere o Rimuovere un nuovo progetto
- O aggiungerne uno già esistente
- Fare la build o il run dei progetti
- **Aggiungere i package da nuget!!!** (o anche rimuoverli)
- E altro ancora…